



Free offer



PSET 1 Review + Project Overview

Ishikaa Lunawat

CS231A

04/11/2025

A bit about me

Hi I'm Ishikaa!

- 2nd year MSEE student (graduating soon, yay!)
- Used to work with Jiajun and Chelsea at SVL
- Interests/Past work:
 - Depth Estimation
 - 3D Scene Reconstruction

Show more ▾

Được. Mình sẽ giải thích **3 bài PSET 1** theo kiểu trực giác trước, công thức sau, vì phần này của CS231A khá dễ bị “ngợp” bởi notation.

Nhìn tổng thể, 3 bài đang đi theo một chuỗi rất logic:

Q1: Hiểu hình học nào được bảo toàn khi biến đổi ảnh



Q2: Dùng các điểm 3D + điểm ảnh để tìm camera



Q3: Chỉ từ một ảnh, suy ra thông tin 3D như hướng, góc, khoảng cách

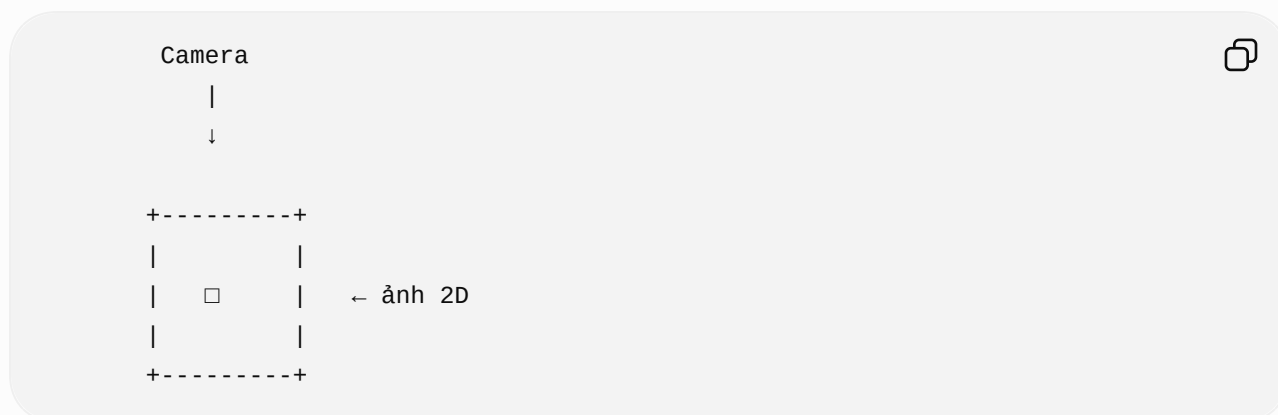
0. Bức tranh lớn trước khi vào từng bài

Hãy tưởng tượng bạn có một vật thể 3D:

```
      P3
      /|
     / |
    P2 |   ← vật thể 3D
    |  |
    |  |
    |  P4
```



Camera chụp nó thành một ảnh 2D:



Vấn đề của Computer Vision là:

Từ ảnh 2D, chúng ta muốn biết lại thông tin 3D.

Ba bài tập lần lượt giúp bạn hiểu:

Bài	Câu hỏi chính
Q1 Projective Geometry	Khi biến đổi hình ảnh, cái gì còn được giữ nguyên?
Q2 Affine Camera Calibration	Biết 3D points và 2D points → tìm camera như thế nào?
Q3 Single View Metrology	Chỉ có một ảnh → suy ra hướng, camera, góc giữa các mặt phẳng như thế nào?

1. Q1 — Projective Geometry

Đây là bài nền tảng hình học.

1.1. Parallel lines là gì?

Ví dụ có hai đường thẳng song song:

k: -----

l: -----



Lấy hai điểm trên mỗi đường:

k_1 ----- k_2



l_1 ----- l_2

Ta có hai đoạn:

$k_1 \rightarrow k_2$



$l_1 \rightarrow l_2$

Vì chúng nằm trên hai đường song song nên hai vector này **cùng phương**.

Do đó:

$$k_1 - k_2 = \lambda(l_1 - l_2)$$

với một số thực λ .

Lấy độ dài:

$$\|k_1 - k_2\| = |\lambda| \|l_1 - l_2\|$$

Do đó:

$$\frac{\|k_1 - k_2\|}{\|l_1 - l_2\|} = |\lambda|$$

1.2. Affine transformation là gì?

Đề cho:

$$A(p) = Mp + b$$

Trong đó:

- p : điểm ban đầu
- M : ma trận biến đổi
- b : translation
- $A(p)$: điểm sau biến đổi

Ví dụ:

Original

After affine transformation



A ----- B

A' ----- B'

Affine transformation có thể bao gồm:

- translation
- rotation
- scaling
- shearing

Ví dụ shear:

Before

After



Điểm quan trọng:

Affine transformation bảo toàn tính song song.

Nếu:

$k \parallel l$



thì sau affine transformation:

$k' \parallel l'$



1.3. Tại sao ratio của hai đoạn song song được bảo toàn?

Đây là phần quan trọng nhất của Q1.

Ta có:

$$k_1 - k_2 = \lambda(l_1 - l_2)$$

Sau affine transformation:

$$k'_1 = Mk_1 + b$$

$$k'_2 = Mk_2 + b$$

Lấy hiệu:

$$k'_1 - k'_2 = (Mk_1 + b) - (Mk_2 + b)$$

Hai b triệt tiêu:

$$k'_1 - k'_2 = M(k_1 - k_2)$$

Tương tự:

$$l'_1 - l'_2 = M(l_1 - l_2)$$

Do:

$$k_1 - k_2 = \lambda(l_1 - l_2)$$

nên:

$$\begin{aligned} k'_1 - k'_2 &= M[\lambda(l_1 - l_2)] \\ &= \lambda M(l_1 - l_2) \\ &= \lambda(l'_1 - l'_2) \end{aligned}$$

Tức là **hệ số λ vẫn giống hệt**.

Vì vậy:

$$\boxed{\frac{\|k'_1 - k'_2\|}{\|l'_1 - l'_2\|} = \frac{\|k_1 - k_2\|}{\|l_1 - l_2\|}}$$

Trực giác

Nếu bạn có:

A --- B



C ----- D

tỷ lệ:

$$\frac{AB}{CD} = \frac{1}{2}$$

Affine transformation có thể kéo dãn toàn bộ hình:

A-----B



C-----D

nhưng vì hai đoạn cùng hướng nên chúng bị biến đổi theo cùng một cách.

Tỷ lệ vẫn giữ nguyên.

1.4. Nhưng tại sao đoạn KHÔNG song song thì không được bảo toàn?

Đây chính là điểm thú vị.

Giả sử:

A--B



C
\
\
D

Hai đoạn có **hướng khác nhau**.

Affine transformation có thể stretch theo một hướng nhiều hơn hướng khác.

Ví dụ:

Before:



horizontal = 10
vertical = 10

Sau scaling:

x × 2
y × 5



```
horizontal = 20
vertical   = 50
```

Ban đầu:

$$\frac{10}{10} = 1$$

Sau biến đổi:

$$\frac{20}{50} = 0.4$$

→ ratio thay đổi.

Ý tưởng cốt lõi của Q1

Affine transformation bảo toàn ratio của các đoạn thẳng cùng phương, nhưng không bảo toàn ratio của các đoạn có hướng khác nhau.

1.5. Projective transformation khác Affine transformation thế nào?

Đây là câu hỏi mà slide muốn bạn hiểu.

Affine:

```
parallel lines
  ↓
parallel lines
```

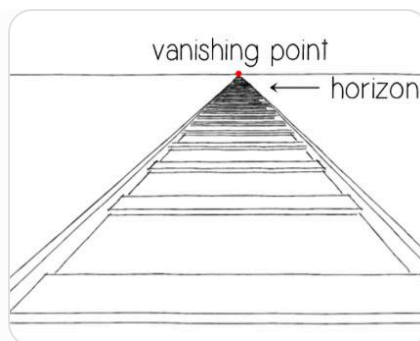
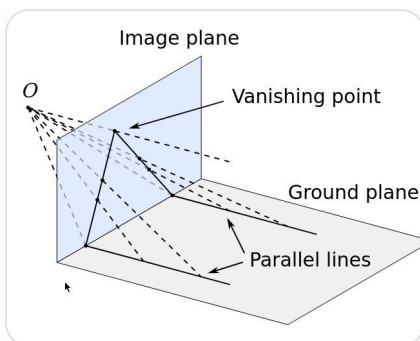
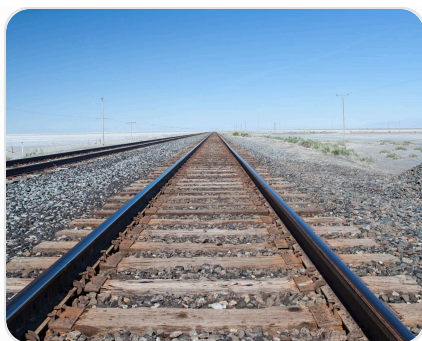


Projective:

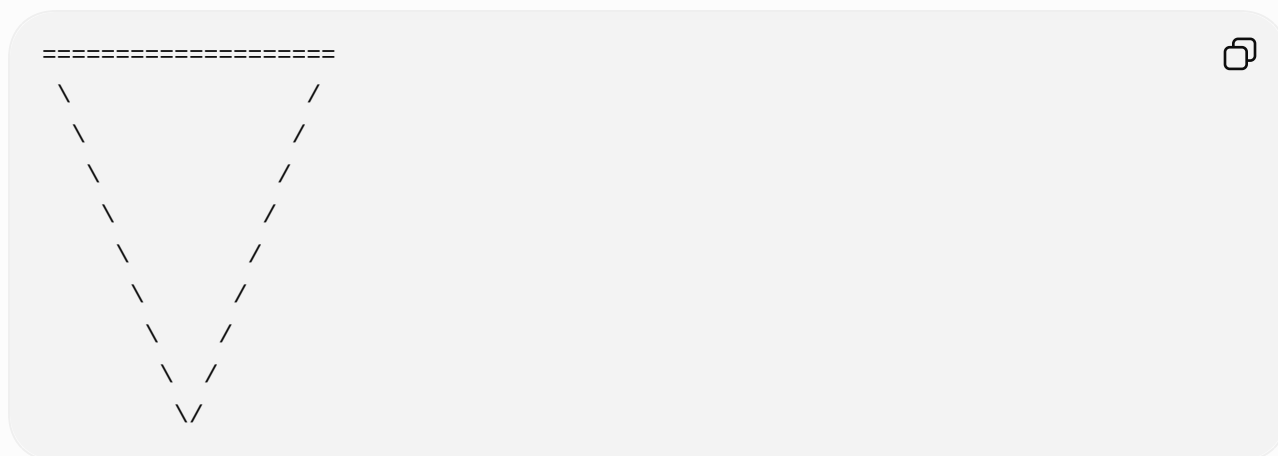
```
parallel lines
  ↓
có thể hội tụ tại một điểm
```



Ví dụ đường ray:



Ngoài đời:



Hai đường ray **thực tế song song**, nhưng trên ảnh chúng hội tụ tại:

Vanishing Point

Đây chính là thứ Q3 sẽ dùng.

2. Q2 — Affine Camera Calibration

Q2 chuyển từ **hình học thuần túy** sang **camera**.

Câu hỏi:

Nếu tôi biết vị trí 3D của các điểm và biết chúng xuất hiện ở đâu trên ảnh, tôi có thể tìm được camera matrix không?

Câu trả lời: **Có**.

2.1. Camera biến 3D thành 2D

Giả sử có một điểm 3D:

$$P = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Camera nhìn nó và tạo ra pixel:

$$p = \begin{bmatrix} x \\ y \end{bmatrix}$$

Trong perspective camera:

$$p \sim KP$$

hoặc đầy đủ hơn:

$$x \sim \frac{fX}{Z}$$

Đây là lý do perspective camera phức tạp:

Z nằm ở mẫu số

2.2. Affine camera đơn giản hóa việc này

Affine camera bỏ qua phần perspective mạnh.

Slide viết:

$$p = MP$$

Trong đó:

$$p = \begin{bmatrix} x \\ y \end{bmatrix}$$

và:

$$P = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Có thể viết:

$$\begin{bmatrix} x \\ y \end{bmatrix} = M \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Trong đó M là ma trận 2×4 :

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \end{bmatrix}$$

Có **8 unknowns**.

Đây chính là slide nói:

Affine Camera Model $\rightarrow$ Linear $\rightarrow$ 8 Unknowns.

2.3. Một điểm 3D cho bạn 2 phương trình

Ví dụ:

$$P_i = (X_i, Y_i, Z_i)$$

và ảnh của nó:

$$p_i = (x_i, y_i)$$

Ta có:

$$x_i = m_{11}X_i + m_{12}Y_i + m_{13}Z_i + m_{14}$$

và:

$$y_i = m_{21}X_i + m_{22}Y_i + m_{23}Z_i + m_{24}$$

Mỗi điểm cho:

2 equations

2.4. Có 8 unknowns thì cần ít nhất 4 points

Ta có:

```
1 point → 2 equations
2 points → 4 equations
3 points → 6 equations
4 points → 8 equations
```



Nên ít nhất:

4 points

để giải 8 unknowns.

Nhưng thực tế thường có **nhiều hơn 4 points**.

Ví dụ:

```
100 3D points
    ↓
100 corresponding image points
    ↓
200 equations
    ↓
8 unknowns
```



Đây là **overdetermined system**.

2.5. Đây là lúc `np.linalg.lstsq` xuất hiện

Bạn gom tất cả thành:

$$Am = b$$

Trong đó:

- A : thông tin về 3D points
- m : 8 unknowns của camera
- b : các pixel coordinates

Sau đó:

`</>` Python



▶ Run

```
m = np.linalg.lstsq(A, b, rcond=None)[0]
```

hoặc:

```
</> Python
```



Run

```
m = np.linalg.pinv(A) @ b
```

2.6. Ví dụ cực đơn giản

Giả sử:

$$x = 2X + 3Y + Z + 5$$

$$y = X + 4Y + 2Z + 7$$

thì camera matrix:

$$M = \begin{bmatrix} 2 & 3 & 1 & 5 \\ 1 & 4 & 2 & 7 \end{bmatrix}$$

Nếu:

$$P = \begin{bmatrix} 10 \\ 20 \\ 30 \\ 1 \end{bmatrix}$$

thì:

$$x = 2(10) + 3(20) + 30 + 5 = 115$$

$$y = 10 + 4(20) + 2(30) + 7 = 127$$

Camera biến:

$$(10, 20, 30)$$

thành pixel:

$$(115, 127)$$

2.7. Sau khi tìm M phải làm gì?

Slide nhấn mạnh:

Reproject the scene points onto the image plane and calculate the error.

Tức là:

Bước 1

Bạn tìm được:

$$M$$

Bước 2

Dùng M dự đoán lại pixel:

$$\hat{p}_i = MP_i$$

Bước 3

So sánh với pixel thật:

$$p_i$$

Bước 4

Tính reprojection error:

$$e_i = \|\hat{p}_i - p_i\|$$

Bước 5

Ví dụ mean error:

$$E = \frac{1}{N} \sum_{i=1}^N \|\hat{p}_i - p_i\|$$

2.8. Q2 thực chất đang dạy bạn điều gì?

Nói đơn giản:

Tôi có một camera và một đồng điểm 3D. Tôi biết các điểm đó xuất hiện ở pixel nào.
Hãy học ngược lại camera matrix.

Đây chính là một bài **linear regression** rất giống **machine learning**:

```
3D points
↓
A
↓
solve
↓
camera parameters M
↓
project
↓
predicted pixels
↓
compare
↓
reprojection error
```



3. Q3 — Single View Metrology

Đây là bài thú vị nhất.

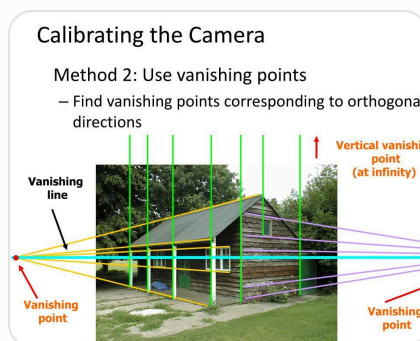
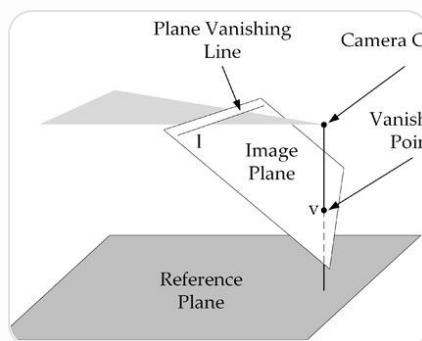
Tên nghe khó:

Single View Metrology

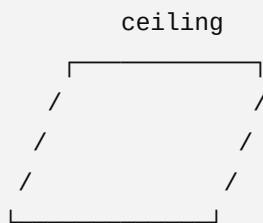
nhưng ý tưởng rất đơn giản:

Chỉ nhìn một bức ảnh, liệu ta có thể suy ra một số thông tin 3D không?

Ví dụ:

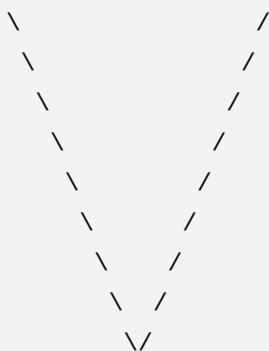


Bạn nhìn một căn phòng:



Các đường thực tế song song.

Nhưng trên ảnh:



Chúng gặp nhau tại một **vanishing point**.

3.1. Vanishing Point là gì?

Giả sử có hai đường:

$$L_1$$

và:

$$L_2$$

Trên ảnh, tìm phương trình của hai đường.

Nếu:

$$L_1 : (x_1, y_1) \rightarrow (x_2, y_2)$$

thì slope:

$$m_1 = \frac{y_2 - y_1}{x_2 - x_1}$$

Tương tự:

$$m_2 = \frac{y_4 - y_3}{x_4 - x_3}$$

Hai đường này giao nhau ở:

V

Đó là **vanishing point**.

3.2. Tại sao vanishing point quan trọng?

Vì nó chứa thông tin về **hướng 3D**.

Giả sử ngoài đời có:

→ → → → →



tất cả các đường đều cùng hướng.

Sau perspective projection:



V



Chúng cùng đi về một vanishing point.

Do đó:

Vanishing Point $\leftrightarrow$ 3D Direction

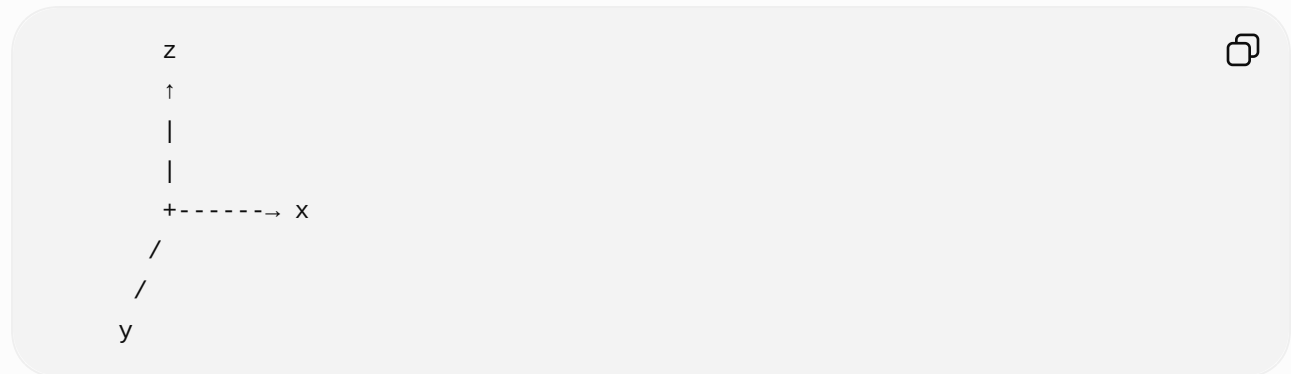
Đây là một trong những ý tưởng quan trọng nhất của Computer Vision.

3.3. Orthogonality — hai hướng vuông góc

Giả sử có ba vanishing points:

$$V_1, V_2, V_3$$

tương ứng với ba hướng:



Nếu các hướng này vuông góc:

$$d_1 \perp d_2$$

thì:

$$d_1^T d_2 = 0$$

Nhưng ta không trực tiếp biết d .

Ta có quan hệ:

$$v = Kd$$

với:

- v : vanishing point
- K : intrinsic camera matrix
- d : hướng 3D

Suy ra:

$$d = K^{-1}v$$

Do đó điều kiện vuông góc trở thành:

$$(K^{-1}v_i)^T (K^{-1}v_j) = 0$$

hay:

$$v_i^T K^{-T} K^{-1} v_j = 0$$

Slide gọi:

$$\omega = K^{-T} K^{-1}$$

hoặc tùy convention có thể viết dạng tương đương.

3.4. Đây là lý do xuất hiện w , A , SVD

Slide nói:

$$Aw = 0$$

với:

$$A : 3 \times 4$$

$$w : 4 \times 1$$

Tại sao?

Vì từ các điều kiện hình học, bạn tạo ra các phương trình tuyến tính liên quan đến các phần tử của:

$$\omega$$

Sau đó gom chúng lại thành:

$$Aw = 0$$

Bạn cần tìm vector w .

3.5. Tại sao dùng SVD?

Bạn phân rã:

$$A = UDV^T$$

SVD cho bạn các singular values.

Bạn lấy:

$$\text{last column of } V$$

làm nghiệm cho w .

Trực giác:

Tìm một vector w sao cho Aw gần bằng zero nhất.

SVD là công cụ rất tốt cho việc này.

3.6. Từ w suy ra K

Sau khi tìm được:

$$w$$

ta muốn tìm:

$$K$$

Slide nói:

Use Cholesky factorization.

Tức là thực hiện Cholesky decomposition để lấy lại intrinsic matrix.

Sau đó:

$$K$$

phải được **normalize**.

3.7. K là gì?

Camera intrinsic matrix thường có dạng:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Trong đó:

- f_x : focal length theo x
- f_y : focal length theo y

- c_x, c_y : principal point
- s : skew

Slide giả sử:

Square pixels

$$f_x = f_y$$

No skew

$$s = 0$$

nên:

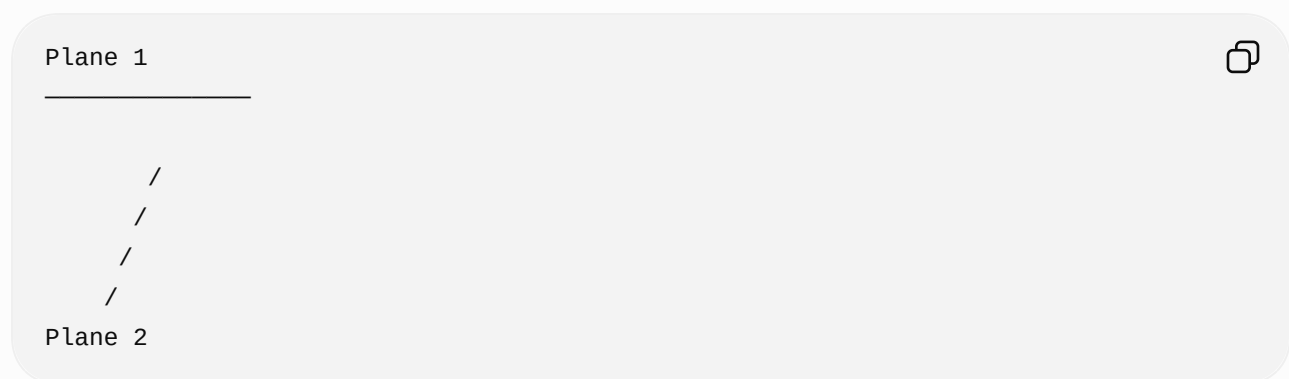
$$K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Điều này giúp giảm số unknowns.

4. Q3 — Compute Angle Between Planes

Đây là phần tiếp theo của Single View Metrology.

Giả sử bạn có hai mặt phẳng:



Bạn muốn biết:

$$\theta = ?$$

4.1. Vanishing line

Một mặt phẳng có thể có hai hướng khác nhau.

Ví dụ mặt sàn:

direction 1 $\rightarrow$ V_1
direction 2 $\rightarrow$ V_2



Hai vanishing points:

$$V_1, V_2$$

Từ đó vanishing line của mặt phẳng:

$$l_1 = V_1 \times V_2$$

Tương tự mặt phẳng thứ hai:

$$l_2 = V_3 \times V_4$$

4.2. Vì sao cross product?

Trong homogeneous coordinates:

Hai điểm:

$$V_1, V_2$$

xác định một đường thẳng:

$$l = V_1 \times V_2$$

Nên:

V_1 ----- V_2

↓ cross product

l = vanishing line



4.3. Angle between planes

Một mặt phẳng có một normal vector:

$$n_1$$

mặt phẳng kia:

$$n_2$$

Góc giữa hai mặt phẳng liên quan đến góc giữa hai normal:

$$\cos \theta = \frac{n_1^T n_2}{\|n_1\| \|n_2\|}$$

Q3 dùng thông tin từ vanishing lines + camera calibration để suy ra các normal này.

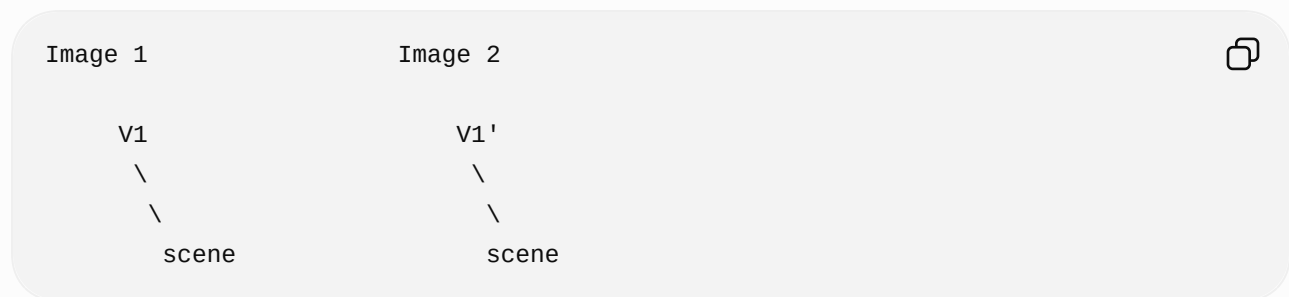
Ý tưởng quan trọng:

Vanishing points $\rightarrow$ directions $\rightarrow$ plane normals $\rightarrow$ angle between planes.

5. P4 — Compute Rotation Matrix

Đây là phần cuối trong slide bạn gửi.

Giả sử bạn có **hai ảnh của cùng một scene**:



Bạn tìm được ba vanishing points:

Image 1:

$$v_1, v_2, v_3$$

Image 2:

$$v'_1, v'_2, v'_3$$

5.1. Từ vanishing point lấy direction

Ta đã biết:

$$v = Kd$$

nên:

$$d = K^{-1}v$$

Do đó:

$$d_i = K^{-1}v_i$$

và:

$$d'_i = K^{-1}v'_i$$

5.2. Camera quay thì direction cũng quay

Giả sử camera chuyển từ Image 1 sang Image 2 bằng rotation:

$$R$$

thì:

$$d'_i = Rd_i$$

Đây chính là câu:

$d'_i = Rd_i$

trong slide.

Nếu biết:

d1 → d1'

d2 → d2'

d3 → d3'

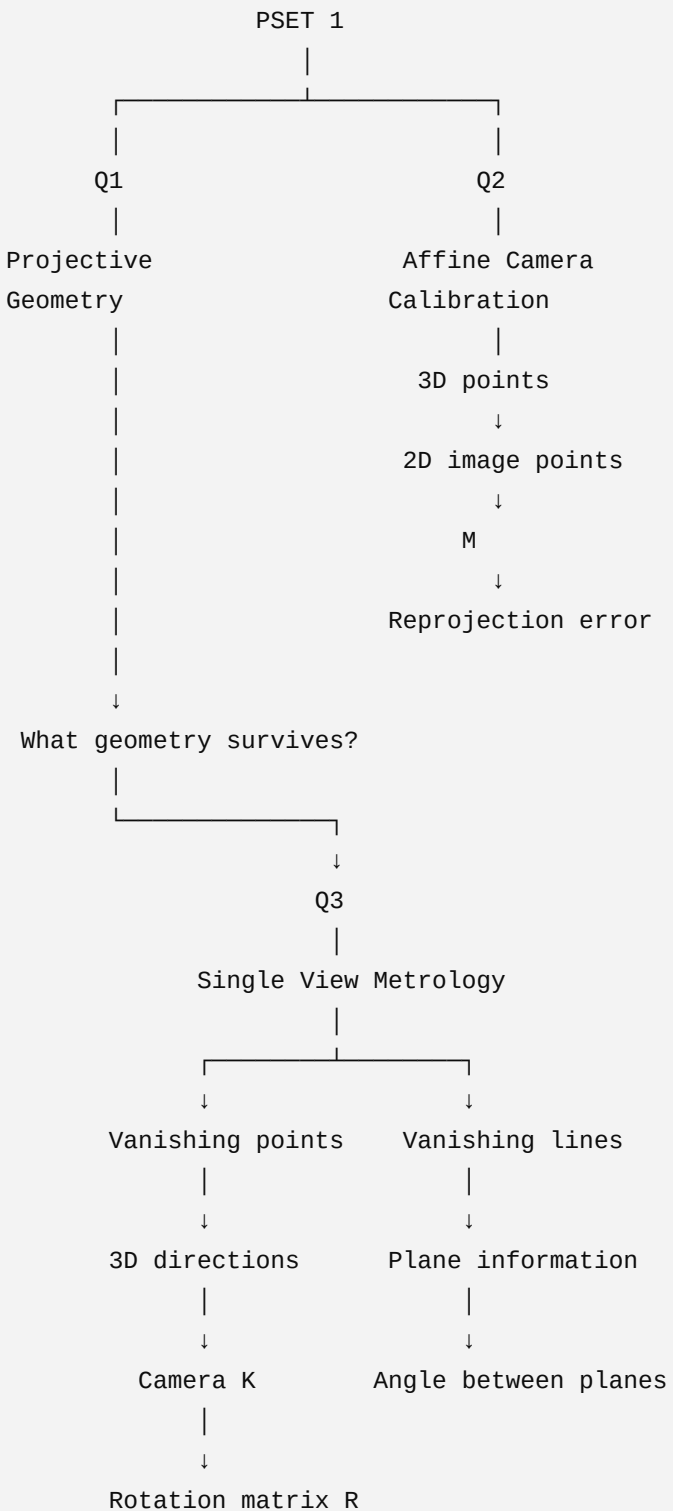


thì ta có thể tìm:

$$R$$

6. Toàn bộ PSET 1 nhìn như một pipeline

Đây là cách mình khuyên bạn ghi nhớ.



7. Cách hiểu Q1 → Q2 → Q3 trong một câu chuyện

Hãy tưởng tượng bạn chụp **một** tòa nhà.

Q1

Bạn hỏi:

"Nếu tôi làm méo ảnh này thì những tính chất hình học nào vẫn đúng?"

→ Affine giữ parallelism và ratio của các đoạn cùng phương.

Q2

Bạn nói:

"Tôi biết tọa độ 3D của một số điểm trên tòa nhà và biết pixel tương ứng. Hãy tìm camera."

→ Tìm:

$$M$$

bằng linear least squares.

Q3

Bây giờ bạn **không muốn biết tọa độ 3D đầy đủ** nữa.

Bạn chỉ có:

một bức ảnh.

Bạn tìm:

```
parallel lines
  ↓
vanishing points
  ↓
3D directions
  ↓
camera intrinsic K
```



↓
plane geometry / angles

Và nếu có **hai ảnh**:

```
vanishing points image 1
  ↓
directions
  ↓
R
  ↑
directions
  ↑
vanishing points image 2
```



→ tìm được **rotation** giữa hai camera.

8. Những thứ bạn thực sự cần nắm để làm PSET

Nếu mục tiêu của bạn là **làm bài/code được**, mình sẽ không cố học thuộc toàn bộ slide.
Hãy tập trung vào các công thức/ý tưởng sau:

Q1

Nhớ:

$$A(p) = Mp + b$$

và:

$$p_1 - p_2 = \lambda(q_1 - q_2)$$

→ affine transformation giữ ratio của **parallel segments**.

Q2

Nhớ:

$$p = MP$$

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \end{bmatrix}$$

→ 8 unknowns.

Sau đó:

$$Am = b$$

→ `np.linalg.lstsq` hoặc `np.linalg.pinv`.

Cuối cùng:

$$E = \frac{1}{N} \sum_i \|\hat{p}_i - p_i\|$$

→ reprojection error.

Q3

Nhớ chuỗi này:

parallel lines → vanishing points → directions

Sau đó:

$$v = Kd$$

nên:

$$d = K^{-1}v$$

Orthogonal directions:

$$d_i^T d_j = 0$$

→ tạo:

$$Aw = 0$$

→ SVD:

$$A = UDV^T$$

→ last column of V

→ w

→ Cholesky

→ K .

9. Một điểm rất quan trọng: đừng nhầm 3 loại matrix

Khi làm PSET, bạn sẽ gặp rất nhiều matrix và dễ nhầm.

Ký hiệu	Ý nghĩa
M	Affine camera matrix trong Q2
K	Camera intrinsic matrix
R	Rotation matrix
A	Matrix của hệ phương trình $Aw = 0$
ω	Image of the absolute conic / $K^{-T}K^{-1}$ trong cách viết của slide
w	Vector chứa các unknowns được giải bằng SVD

Đặc biệt:

$$M \neq K \neq R$$

10. Nếu phải học theo thứ tự

Mình khuyên bạn học theo thứ tự này:

Bước 1: Hiểu homogeneous coordinates và cross product.

Bước 2: Hiểu affine vs projective transformation.

Bước 3: Hiểu camera projection:

$$3D \rightarrow 2D$$

Bước 4: Làm Q2 trước.

Vì Q2 tương đối thẳng:

```
3D points
↓
linear equations
↓
least squares
↓
M
↓
reprojection
```



Bước 5: Sau đó mới học Q3:

```
parallel lines
↓
vanishing points
↓
K
↓
3D directions
↓
plane / angle
↓
rotation
```



Q3 khó hơn Q2 chủ yếu vì nó bắt bạn **đi ngược từ 2D → 3D geometry**, thay vì chỉ làm một hệ phương trình tuyến tính đơn giản.

Nếu bạn đang chuẩn bị **code PSET này**, mình có thể tiếp tục giải thích theo kiểu **“mỗi câu hỏi phải code gì, input/output là gì, ma trận A được xây như thế nào, và cho một ví dụ NumPy hoàn chỉnh cho Q1 → Q2 → Q3”**.

